Unit- II

Syllabus for Examination:
Classes and objects: Definition of classes – access specifier – class versus structure – constructor – destructor – copy constructor and its importance

Unit-III
Syllabus for Examination:
Polymorphism- Compile-time Polymorphism- Function Overloading

Class and Objects

What is a Class?

A class is a user-defined data type.
It is like a blueprint used to create objects.

A class contains:

Attributes → variables
Methods → functions

These together are called class members.

✅ **What is an Object?**

An object is a variable created from a class.
It is a real-world entity having:

Properties (attributes)
Behaviors (methods)

Example:
A Car is an object → it has color, model, speed, and functions like brake().

⭐ Simple Example of a Class

```
class MyClass {
  public:
    int myNum;
    string myString;
};
```

class → used to define a class.
MyClass → name of the class.
public → members accessible from outside.
myNum, myString → attributes of the class.
Class ends with a semicolon (;).

Creating an Object:

```
int main() {
  MyClass myObj;    // object creation

  myObj.myNum = 15;
  myObj.myString = "Some text";

  cout << myObj.myNum << "\n";
  cout << myObj.myString;
}
```

Note: We access members using the dot operator (.).

⭐ **Multiple Objects in a class**

```
class Car {
  public:
    string brand;
```

```
    string model;
    int year;
};

int main() {
  Car c1;
  c1.brand = "BMW";
  c1.model = "X5";
  c1.year  = 1999;

  Car c2;
  c2.brand = "Ford";
  c2.model = "Mustang";
  c2.year  = 1969;

  cout << c1.brand << " " << c1.model << " " << c1.year << endl;
  cout << c2.brand << " " << c2.model << " " << c2.year<<endl;
}
```

Access Modifiers in C++ Class:

Access modifiers control who can access class members.
There are three types:

1. Public

Members can be accessed from anywhere.
Most commonly used for functions.

2. Private

Members can be accessed only inside the class.
Used for data hiding.
Default access level in a class.

Example

```
class Employee
{
private:
   double salary;
   int empID;

public:
   string name;

   Employee(string n, double s, int id)
{
       name = n;
       salary = s;
       empID = id;
   }
};
```

3. Protected

Accessible:
✓ inside the class
✓ inside derived (child) classes
Not accessible from outside the class.

Example with Inheritance

```
class Employee
{
private:
    double salary;
```

```
protected:
    int empID;

public:
    string name;

    Employee(string n, double s, int id
{
}
};

class Manager : public Employee
{
public:
    Manager(string n, double s, int id)
    void showDetails() {
        cout << "Manager Name: " << name << endl;
        cout << "Manager ID: " << empID << endl;    // allowed, empID
is protected
    }
};
```

🎯 Summary to Write in Exams

A class is a blueprint; an object is an instance of the class.
Class members include attributes (variables) and methods (functions).
Objects access class members using the dot operator.
Access Modifiers:

public → accessible everywhere
private → only inside class
protected → inside class + derived classes

Constructors and Destructors in C++

Constructor (Simple Definition)
A constructor is a special function inside a class that runs automatically when an object is created.
It initializes the object.

Key Points

- ✓ Same name as the class
- ✓ No return type (not even void)
- ✓ Can be public, private, or protected
- ✓ Called automatically at object creation
- ✓ Can have parameters
- ✓ Cannot be virtual

⋆ Types of Constructors

1. Default Constructor

A constructor with no parameters.
Example

```
#include <iostream>
using namespace std;

class Person
{
public:
   Person()
{
```

```
        cout << "Default constructor called\n";
    }
};

int main() {
    Person p;   // default constructor runs
}
```

2. Parameterized Constructor

A constructor with parameters, used to set values at object creation.

Example

```
#include <iostream>
using namespace std;

class Person
{
    string name;
    int age;
public:
    Person(string n, int a) {
        name = n;
        age = a;
    }
    void display() {
        cout << name << " " << age;
    }
};

int main() {
    Person p("Hari", 20);
```

```
    p.display();
}
```

3. Copy Constructor
Creates a new object as a copy of an existing object.
Syntax
C++ClassName(const ClassName &obj)

Example

```
#include <iostream>
using namespace std;

class Person
{
    int age;
public:
    Person(int a)
 {
        age = a;
    }
    Person(const Person &p)
{
        age = p.age;
    }
    void show()
{
cout << age;
}
};

int main() {
    Person p1(25);
    Person p2(p1);  // copy constructor
    p2.show();
}
```

Destructor (Simple Definition)

A destructor is a special function that runs automatically when an object is destroyed.
It is used to free memory.

Key Points

- ✓ Same name as class but starts with ~
- ✓ No return type, no parameters
- ✓ Called automatically when:

- ✓ Object goes out of scope
- ✓ Program ends

- ✓ Cannot be overloaded

Destructor Example

```
#include <iostream>
using namespace std;

class Test
{
public:
   Test()
{
cout << "Constructor called\n";
 }
   ~Test()
{
cout << "Destructor called\n";
}
```

```
};

int main() {
    Test t;  // constructor runs
}           // object destroyed → destructor runs
```

Order of Execution

When an object is created:

☞ Constructor → Program Code → Destructor

Summary:

→ Constructors initialize objects; destructors clean them up.
→ Constructor = same name as class; Destructor = ~classname().
→ Constructors can be default, parameterized, copy, or dynamic.
→ Destructors have no parameters and cannot be overloaded.
→ Constructor is called at creation; destructor is called at destruction.

Programs for Practise:

Problem: Area of a rectangle

Write C++ programs using the following concepts:

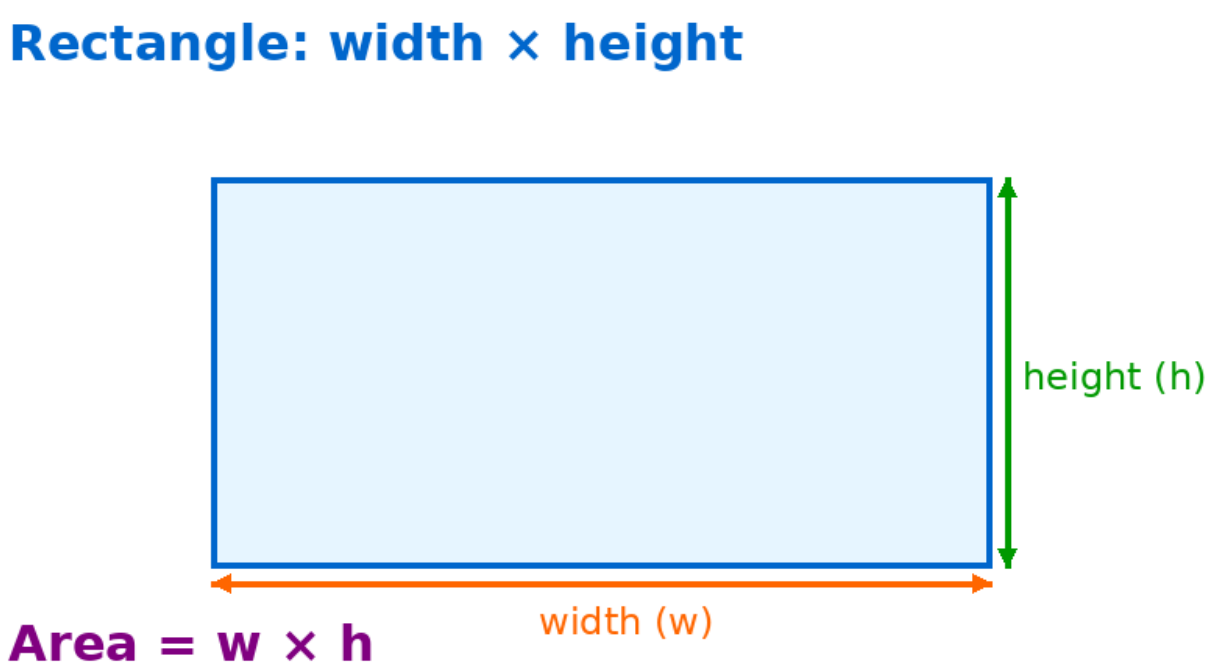


Figure: A rectangle with width (w) and height (h). Area = w × h.

1) Function (No Parameters)

- The function asks for width and height.
- It returns w × h.
- Main prints the answer.

Code:

```
#include <iostream>
using namespace std;

float area() {
    float w, h;
    cout << "Enter width: ";
    cin >> w;
    cout << "Enter height: ";
    cin >> h;
    return w * h;
}

int main() {
    cout << "Area = " << area();
    return 0;
```

```
}
```

2) Function With Parameters

• We send width and height values from main program to the function.

• The function multiplies them and returns the area to main through parameters.

Code:

```
#include <iostream>
using namespace std;

float area(float w, float h) {
    return w * h;
}

int main() {
    float w, h;
    cout << "Enter width: ";
    cin >> w;
    cout << "Enter height: ";
    cin >> h;

    cout << "Area = " << area(w, h);
    return 0;
}
```

3) Call By Value (Copy)

• A copy of w and h is made.

• Changes inside the function do not change the original values in the main prgram.

Code:

```
#include <iostream>
using namespace std;

float areaValue(float w, float h) {
    w = w + 1;
    h = h + 1;
    return w * h;
```

```
}

int main() {
    float w = 4, h = 3;
    cout << "Area = " << areaValue(w, h) << "
";
    cout << "Original width = " << w << ", height = " << h;
    return 0;
}
```

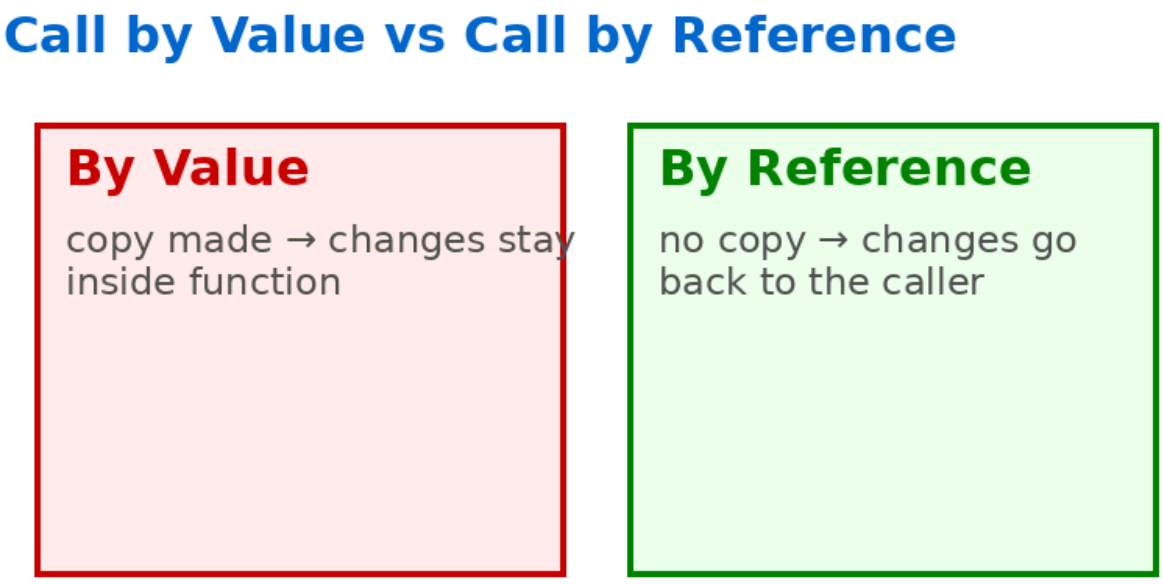


Figure: Call by Value makes a copy; Call by Reference shares the same variable.

4) Call By Reference (Shares back the value to main function)

• No copy is made; we use & to send a reference.
• The function can write the result back.

Code:

```
#include <iostream>
using namespace std;

void areaRef(float w, float h, float &result) {
    result = w * h;
}
```

```
int main() {
    float w, h, a;
    cout << "Enter width: ";
    cin >> w;
    cout << "Enter height: ";
    cin >> h;

    areaRef(w, h, a);
    cout << "Area = " << a;
    return 0;
}
```

5) Class

• A class groups data (width, height) and actions (area).

• We make an object and call its area() method.

Code:

```
#include <iostream>
using namespace std;

class Rectangle
{
public:
    float width, height;

    float area()
{
        return width * height;
    }
};

int main() {
    Rectangle r;
    cout << "Enter width: ";
    cin >> r.width;
    cout << "Enter height: ";
    cin >> r.height;
```

```
    cout << "Area = " << r.area();
    return 0;
}
```

6) Inline Function

• inline suggests the compiler to replace the call with the function body.

• It's just a hint; compilers decide what to do.

Code:

```
#include <iostream>
using namespace std;

inline float area(float w, float h)
{
    return w * h;
}

int main() {
    float w, h;
    cout << "Enter width: ";
    cin >> w;
    cout << "Enter height: ";
    cin >> h;

    cout << "Area = " << area(w, h);
    return 0;
}
```

7) Constructor & Destructor

• Constructor runs when an object is created (we set w and h).

• Destructor runs when an object is destroyed (goodbye message).

Code:

```
#include <iostream>
using namespace std;

class Rectangle {
public:
```

```
    float w, h;

    Rectangle(float width, float height) {
        w = width;
        h = height;
        cout << "Object Created
";
    }


    ~Rectangle() {
        cout << "Object Destroyed
";
    }

    float area() {
        return w * h;
    }
};

int main() {
    Rectangle r(5, 3);
    cout << "Area = " << r.area() << "
";
    return 0;
}
```

8) Function Overloading (Same Name)

• Two functions named area: one for int, one for float.

• The compiler chooses based on the argument types.

Code:

```
#include <iostream>
using namespace std;

int area(int w, int h) {
    return w * h;
}
```

```
float area(float w, float h) {
    return w * h;
}

int main() {
    cout << "Area int = " << area(4, 5) << "
";
    cout << "Area float = " << area(4.5f, 2.5f) << "
";
    return 0;
}
```

Polymorphism in C++

1. Definition

Polymorphism in C++ means **one function behaving in different ways** depending on the object or data used.
In simple words: **same name → different work**.

Types of Polymorphism in C++

C++ has **two types**:

1. Compile-Time Polymorphism

Happens **during compilation**.
Achieved using:

- **Function Overloading**
- **Operator Overloading**

2. Runtime Polymorphism

Happens **while the program is running**.
Achieved using:

- **Function Overriding** (with virtual functions)

Compile-Time Polymorphism Example: Function Overloading

What is Function Overloading?

- Same function name
- Different parameter types or count
- Compiler decides which one to call

Example (simple):

C++
void show(int a);
void show(double b);
void show(string s);
Show more lines

Here, show() functions have the **same name but different parameters**, so C++ picks the correct one based on the argument.

```
#include <iostream>
using namespace std;

class Overload {
public:
    void print(int num) {
        cout << "Printing integer: " << num << endl;
    }

    void print(double num) {
        cout << "Printing double: " << num << endl;
    }

    void print(string text) {
```

```
        cout << "Printing string: " << text << endl;
    }
};

int main() {
    Overload obj;

    obj.print(5);
    obj.print(10.5);
    obj.print("Hello World");

    return 0;
}
```

Output

```
1
2
3
Printing integer: 5
Printing double: 10.5
Printing string: Hello World
```

Why Polymorphism is Useful?

- Makes code **flexible**
- Allows **reusability**
- Reduces **complexity**
- Helps implement **OOP concepts** cleanly

One-Line Real-Life Example (for exam):

A person behaves differently as a **student, friend, or employee** → one person, many behaviours.
Similarly, in C++, one function name can perform many tasks.